Technical Report on the Migration of `chess.bc.ca` to a Git Content Management System

British Columbia Chess Federation

July 2026

Executive Summary

In July 2026 the British Columbia Chess Federation's website, https://chess.bc.ca, was rebuilt from the ground up and moved to a new hosting arrangement. Visitors see a modernized version of the same site — news, clubs, events, bulletins, champions, and so on — but behind the scenes everything has changed.

The old site was a collection of hand-edited web pages on a commercial web server, costing about $150 per year. Updating it required editing raw HTML/PHP code and uploading files to the server, so in practice only one person could maintain it.

The new site works differently:

- **It costs nothing to host.** The site is served by Netlify's free tier, eliminating the roughly $150/year hosting fee. The only remaining cost is the annual domain-name registration for `chess.bc.ca`.
- **Anyone authorized can edit it.** Editors log into a simple web form at https://chess.bc.ca/admin/ — no coding knowledge, no special software. Posting a news item is as easy as filling out a form and clicking *Publish*.
- **It looks good on phones.** The layout automatically adapts to any screen size.
- **It largely runs itself.** News items automatically move to the archive when they expire, the site rebuilds itself every night, and every change ever made is permanently recorded and reversible.

This report is written for the person who will maintain the site. It assumes familiarity with basic web technologies and GitHub, but not with the other tools involved. It explains what each piece of the system does, how the pieces fit together, and how to perform routine maintenance.

Contents

1 Introduction

The previous `chess.bc.ca` was a PHP website: each time a visitor requested a page, a program on the web server assembled the page from fragments of PHP and HTML. There was no database and no administrative interface; all changes were made by editing code files directly on the server. The site predated smartphones and did not adapt to small screens.

The replacement is a *static site* managed through Git. All content lives as plain text files in a GitHub repository. A tool called Hugo converts those files into ordinary HTML pages, and Netlify hosts the result on a global content-delivery network. Editors never touch the code: a web-based editing interface (Decap CMS) reads and writes the same repository through friendly forms.

Table 1 summarizes the change.

	Old site	**New site**
Pages built	at view time, by PHP	in advance, by Hugo
Source of truth	files on the web server	GitHub repository
Editing	hand-edit code, upload	web forms at `/admin/`
Editors	one (the webmaster)	any invited editor
Hosting cost	$\approx$ \$150/year	\$0 (Netlify free tier)
Mobile support	none	fully responsive
Change history	none	every change is a Git commit
HTTPS certificate	managed by host	automatic (Let's Encrypt)

Table 1: The migration at a glance.

2 Benefits of the Migration

1. **No server-side scripting; free hosting.** The site no longer uses a scripting language to build pages at view time. Instead, the site rebuilds and uploads itself automatically whenever content changes. Because the result is plain files, it can be hosted on a service that is free for a small non-profit organization. There are no more server costs — a saving of about \$150 per year. The only remaining recurring cost is the yearly domain-name registration.

2. **Responsive design.** The site's appearance now automatically adjusts to the screen size. On a phone, the navigation collapses into a menu button and content reflows into a single readable column; on a desktop, the full layout appears. The site is now comfortable to read on any device.

3. **Structured content, editable by anyone.** The site now uses a content management system that structures the data behind its various lists: clubs, repeating events, chess instructors, bulletins, arbiters, champions, links, and banners. All of these can be created and edited through simple web forms. Previously, modifying the website

required an understanding of web coding; now anyone with an editor account can keep the site current.

4. **New logo.** The site carries the Federation's new logo — the chessboard map of British Columbia — on every page, replacing the old banner image.
5. **Automatic news archiving.** Old news moves to the News Archive automatically after an expiry period, which defaults to two weeks from the posting date. Editors can override the expiry date per item, or simply leave the field blank and accept the default.
6. **Rotating banner ads.** The banner advertisements are shuffled into a random order at every site build — at least daily — so no advertiser is permanently stuck at the bottom.

Beyond the requested items above, the migration also delivers:

7. **A complete, reversible change history.** Every edit — whether made by an editor in the web forms or by a developer in code — becomes a Git commit with an author and timestamp. Any mistake can be undone by reverting the commit, and nothing is ever silently lost.
8. **Stronger security and reliability.** With no PHP, no database, and no server of our own, there is essentially nothing to hack and nothing to patch. The pages are plain files distributed across a global content-delivery network, which also makes the site fast everywhere. If a build ever fails, Netlify simply keeps serving the last good version — a bad edit cannot take the site offline.
9. **HTTPS everywhere, automatically.** The site enforces encrypted connections with a free Let's Encrypt certificate that renews itself; no one has to remember to renew it.
10. **Old links keep working.** Every address from the old site (`/clubs.php`, `/Bulletins/....pdf`, and so on) permanently redirects to its new equivalent, preserving bookmarks, search rankings, and links from other websites.
11. **Individual editor accounts.** Editors are invited by email and each has a personal login. There are no shared passwords, access can be revoked per person, and each person's edits are attributed to them in the site's history.
12. **Portability.** The repository contains everything — content, templates, styling, and configuration. If Netlify ever became unsuitable, the site could be rebuilt and hosted elsewhere without loss.

3 The Technology Stack

The system has four main components plus one small helper. Each is described below for a reader who has not used it before.

3.1 GitHub — the source of truth

Everything that defines the website lives in one Git repository:

https://github.com/stmorgan/chess.bc.ca-hugo

This includes the content (news items, club listings, ...), the page templates, the stylesheet, all uploaded images and bulletin PDFs, and the configuration for every other tool in the stack. The `main` branch *is* the website: whatever is committed there is what gets published.

3.2 Hugo — the static site generator

Hugo (https://gohugo.io) is a program that turns the repository's content files into a complete website. Content is written in Markdown files with a small block of metadata ("front matter") at the top — a title, a date, and so on. Hugo pours that content into HTML templates (in the `layouts/` directory), applies the site's CSS, and writes out plain HTML pages. It also performs build-time logic such as filtering expired news onto the archive page and shuffling the banner order. A full build of the site takes under a second.

Hugo runs *at build time only*. Visitors receive finished HTML; nothing is computed when a page is viewed. The Hugo version is pinned in `netlify.toml` so builds are reproducible.

3.3 Netlify — hosting, builds, DNS, and logins

Netlify (https://www.netlify.com) is a hosting platform for static sites. It plays five distinct roles here, which is why it deserves the longest explanation:

Hosting/CDN. Netlify serves the built site from data centres around the world under the project name `chess-bc-ca`.

Continuous deployment. Netlify watches the GitHub repository. Every push to `main` — whether from a developer or from the CMS — triggers Netlify to check out the repository, run Hugo, and publish the result. A typical edit is live within a minute or two.

DNS. The domain's nameservers point at Netlify DNS, so Netlify answers "where is `chess.bc.ca`?" for the whole internet. The domain-name *registration* itself stays with the Federation's registrar; only the yearly renewal is paid there.

HTTPS. Netlify obtains and automatically renews the site's Let's Encrypt certificate, and forces all traffic onto HTTPS.

Identity and Git Gateway. These two features power the editor login system, described with Decap CMS below.

Configuration that belongs to the site itself — the build command, redirects from old URLs, and the canonical-domain redirect — lives in `netlify.toml` at the repository root, so it is versioned along with everything else.

3.4 Decap CMS — the editors' web forms

Decap CMS (https://decapcms.org) provides the editing interface at https://chess.bc.ca/admin/. It is not a server: it is a single-page application served as part of the website itself. Its configuration file, `static/admin/config.yml`, defines every collection an editor sees — which fields a news item has, where club files are stored, which lists live in which data file.

When an editor saves an entry, Decap does not upload anything to a server of its own. Instead it writes a Git commit to the GitHub repository. Because editors should not need GitHub accounts, the commit travels through two Netlify features:

Netlify Identity manages the editor accounts. Registration is *invite-only*: an administrator invites an editor by email from the Netlify dashboard, the editor sets a password, and thereafter logs in at `/admin/`.

Git Gateway is a bridge that lets a logged-in Identity user commit to the repository without having a GitHub account. It authenticates the editor and performs the commit on their behalf.

The Decap application file itself is *self-hosted* — it is committed to the repository at `static/admin/decap-cms.js` (version 3.14.1) rather than loaded from a third-party CDN. This makes the admin site immune to CDN outages and means the CMS only changes when we deliberately replace that file with a newer release.

3.5 GitHub Actions — the nightly rebuild

News expiry and banner shuffling are decisions made *at build time*. If nothing were ever rebuilt, an expired news item would sit on the home page forever, because time passing does not rebuild the site by itself. A small scheduled job, defined in `.github/workflows/nightly-rebuild.yml`, therefore fires every night at 2:17 a.m. Pacific time and pings a Netlify *build hook* (a secret URL that triggers a build). The hook URL is stored as a GitHub Actions secret named `NETLIFY_BUILD_HOOK`. This nightly build is what actually moves news into the archive on its expiry date and reshuffles the banners each day.

3.6 External embeds

Two pieces of the site are embedded from outside services and involve no maintenance: the events calendar on the Events page is an embedded Google Calendar (edited in Google Calendar itself, not the CMS), and the daily puzzle on the home page is embedded from Lichess.

4 How the Pieces Interact

An editor publishes a news item. The editor logs into `/admin/`, clicks *News* → *New News Item*, fills in the form, and clicks *Publish*. Decap (via Git Gateway) commits a new Markdown file to `content/news/` on GitHub. Netlify notices the push, runs Hugo, and

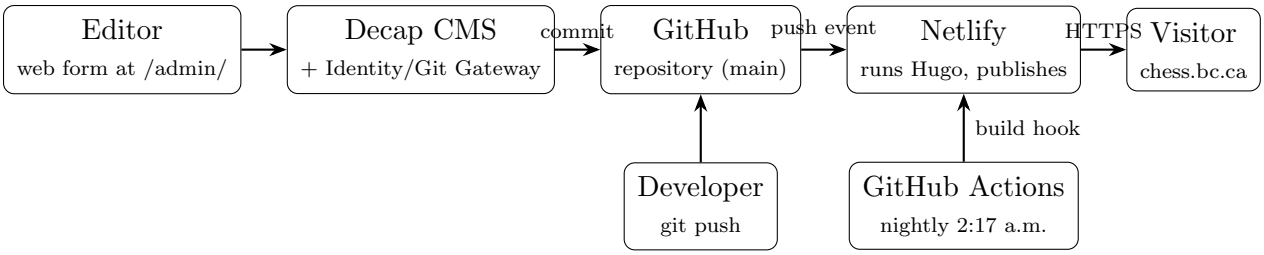

Figure 1: Every path to the published site converges on the same pipeline: a commit (or build trigger) causes Netlify to run Hugo and publish.

publishes the updated site. The item appears on the home page within a minute or two, and will move itself to the News Archive after its expiry date.

A developer changes a template. The developer edits files locally, previews with `hugo server`, commits, and pushes to `main`. The same Netlify pipeline builds and publishes. If the build fails, the previous version keeps serving and the deploy log (Netlify dashboard → *Deploys*) shows the error.

Nothing happens all day. At 2:17 a.m. the GitHub Actions workflow pings the build hook; Netlify rebuilds the unchanged repository. Any news items whose expiry date has passed drop off the home page, and the banners reshuffle.

5 Repository Tour

Two details of the content model are worth knowing before editing files by hand:

- Every file in a folder collection (news, clubs, instruction, bulletins) carries a hidden front-matter field `type` equal to its section name. The CMS uses it to keep Hugo's section index files out of the editors' entry lists. *Keep this field* when creating files manually.
- News expiry: if `expiry_date` is absent, templates fall back to the posting date plus 14 days. The CMS fills the field automatically at save time when an editor leaves it blank.

6 Routine Maintenance

6.1 Inviting or removing editors

Netlify dashboard → project `chess-bc-ca` → *Identity* → *Invite users*. The invitee receives an email, sets a password, and can immediately edit at `/admin/`. To revoke access, delete the user from the same screen. Registration is set to invite-only; do not change this, or anyone on the internet could sign themselves up as an editor.

Path	Contents
content/news/	one Markdown file per news item
content/clubs/	one file per club (grouped by city on the page)
content/instruction/	one file per instructor/academy listing
content/bulletins/	one file per bulletin issue (metadata only)
content/*.md	standalone pages (About, Membership, ...)
data/*.yaml	list data: arbiters, links, champions, banners, repeating events
layouts/	Hugo HTML templates and partials
static/css/style.css	the hand-written responsive stylesheet
static/admin/	Decap CMS: index.html, config.yml, the vendored decap-cms.js
static/bulletins/	all bulletin PDFs
static/images/uploads/	images uploaded through the CMS
static/files/, static/AGM/, ...	documents carried over from the old site
hugo.toml	Hugo configuration and navigation menus
netlify.toml	build command, Hugo version, all redirects
.github/workflows/	the nightly rebuild workflow
docs/	this report

Table 2: Repository layout.

6.2 Local development

1. Clone the repository and install Hugo (extended edition).
2. Run `hugo server` and browse the printed local address. Edits to files rebuild the preview instantly.
3. To try the CMS locally without logging in, additionally run `npx decap-server` and open `/admin/` on the local site. (This works because `config.yml` sets `local_backend: true`, which has no effect in production.)
4. Before pushing, run a full `hugo` build to catch errors that the development server can mask.

6.3 Upgrading the pinned versions

- **Hugo**: change `HUGO_VERSION` in `netlify.toml`, test locally with the same version, push.
- **Decap CMS**: download a newer `dist/decap-cms.js` from the Decap releases and replace `static/admin/decap-cms.js`. Do this deliberately and test the admin afterwards; the file is pinned precisely so the CMS cannot change underneath the editors.

6.4 DNS, domain, and certificate

The domain registration renews yearly at the registrar — this is the site's only recurring cost. The registrar delegates to Netlify DNS (nameservers `dns1--dns4.p05.nsone.net`);

DNS records, should any ever be needed (for example MX records if the Federation adopted `@chess.bc.ca` email), are managed in the Netlify dashboard under *Domains*. The HTTPS certificate renews itself; no action is ever required.

6.5 Costs

- Hosting, builds, DNS, HTTPS, editor logins: $0 on Netlify's free tier. The site's traffic and build volume sit far below the free allowances.
- Domain registration: the usual yearly fee at the registrar.
- GitHub: free for public repositories.

7 Troubleshooting

An editor's form comes up blank, or the admin misbehaves. Have them hard-refresh the browser (Ctrl+Shift+R) to shed a stale cached copy of the CMS, and if that fails, log out and back in (a stale login token produces exactly this symptom).

A change was published but the site did not update. Check the Netlify dashboard → *Deploys*. A red "Failed" deploy means the build broke; open it to read the Hugo error. The site keeps serving the last good version meanwhile.

Expired news is not moving to the archive. The nightly rebuild has probably stopped. Check the repository's *Actions* tab on GitHub: the "Nightly Netlify rebuild" workflow should show nightly runs. Note that GitHub automatically pauses scheduled workflows in repositories with no commits for 60 days (it emails a warning first); any push, or pressing *Run workflow*, resumes it. Also confirm the `NETLIFY_BUILD_HOOK` secret still matches a valid build hook in Netlify.

Something was deleted or broken by mistake. Nothing is lost. Find the offending commit in the repository history on GitHub and revert it, or restore the previous version of the file. The next push republishes the corrected site. Netlify's *Deploys* list also allows one-click "publish deploy" rollback to any earlier build.

A mangled entry breaks the layout below it. The site allows raw HTML inside content (editors' rich text needs it), so an unterminated tag pasted into an entry can visually swallow what follows. Edit the entry and remove the stray fragment. This occurred once during migration (a dangling `</p` in a club listing inherited from the old site).

8 Key Reference

Live site	https://chess.bc.ca
Editing interface	https://chess.bc.ca/admin/
Repository	https://github.com/stmorgan/chess.bc.ca-hugo
Netlify project	`chess-bc-ca` (dashboard: app.netlify.com)
Hugo version	0.163.3 extended (pinned in `netlify.toml`)
Decap CMS version	3.14.1 (vendored at `static/admin/decap-cms.js`)
Nightly rebuild	GitHub Actions, 10:17 UTC (2:17 a.m. Pacific)
Nameservers	`dns1.p05.nsone.net` … `dns4.p05.nsone.net`
Content at migration	99 news items, 45 clubs, 11 instruction listings, 485 bulletins, 234 champions entries
Migration completed	July 8, 2026 (DNS cutover)

Table 3: Quick reference.